ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

| старший преподаватель | | С.А. Рогачев |
|---|---|---|
| должность, уч. степень, звание | подпись, дата | инициалы, фамилия |

# ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

# «СТЕК И ОЧЕРЕДЬ»

по курсу: АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ

РАБОТУ ВЫПОЛНИЛ

| СТУДЕНТ ГР. № | 4236 | | А.В. Панченко |
|---|---|---|---|
| | | подпись, дата | инициалы, фамилия |

Санкт-Петербург 2023

## 1)Цель работы:

Реализовать структуры данных «стек» и «очередь» в соответствии с заданным вариантом. Дополнительно программа должна удовлетворять следующим требованиям:

1) Вывод на экран состояния моделируемой системы на каждой итерации работы (содержимое стека(ов), очереди(ей), процессора(ов));

2) Для каждой задачи из списка входных задач должно быть определено время поступления;

3) Необходимо наличие, как автоматического генератора задач, так и возможность ручного добавления задач, с указанием их параметров (в зависимости от задания);

4) Необходимо обработать ситуации, при которых какая-либо структура данных может быть переполнена

### Вариант №1

| № вар. | Номер задачи | Реализация стека и очереди |
|---|---|---|
| 1 | 1 | Стек – статический; очередь – статическая |

## 2)Листинг программы:

```
#include <iostream>
#include <chrono>
#include <string>
#include <vector>
#include <mutex>
#include <random>
#include <thread>

using namespace std;

const int MaxTasks = 3; // Максимальное количество задач в каждой очереди и стеке

struct Task {
    int priority;
    string name;
    chrono::time_point<chrono::steady_clock> arrivalTime;

    Task(int p, const string& n, chrono::time_point<chrono::steady_clock> time)
        : priority(p), name(n), arrivalTime(time) {}
};

mutex mtx;

void displayState(
```

```
    const vector<Task>& f0,
    const vector<Task>& f1,
    const vector<Task>& f2,
    const vector<Task>& S,
    const Task& currentTask,
    chrono::time_point<chrono::steady_clock> startTime
) {
    chrono::time_point<chrono::steady_clock> currentTime =
chrono::steady_clock::now();
    chrono::duration<double> elapsedTime = currentTime - startTime;

    cout << "Очередь f0 (высший приоритет): ";
    for (const Task& task : f0) {
        chrono::duration<double> taskArrivalTime = task.arrivalTime - startTime;
        cout << task.name << "(" << taskArrivalTime.count() << "s) ";
    }
    cout << "\n";

    cout << "Очередь f1 (средний приоритет): ";
    for (const Task& task : f1) {
        chrono::duration<double> taskArrivalTime = task.arrivalTime - startTime;
        cout << task.name << "(" << taskArrivalTime.count() << "s) ";
    }
    cout << "\n";

    cout << "Очередь f2 (самый низкий приоритет): ";
    for (const Task& task : f2) {
        chrono::duration<double> taskArrivalTime = task.arrivalTime - startTime;
        cout << task.name << "(" << taskArrivalTime.count() << "s) ";
    }
    cout << "\n";

    cout << "Стек S: ";
    for (const Task& task : S) {
        chrono::duration<double> taskArrivalTime = task.arrivalTime - startTime;
        cout << task.name << "(" << taskArrivalTime.count() << "s) ";
    }
    cout << "\n";

    if (currentTask.priority >= 0) {
        chrono::duration<double> taskArrivalTime = currentTask.arrivalTime -
startTime;
        cout << "Выполняется задача: " << currentTask.name << " (приоритет " <<
currentTask.priority << ", поступила через " << taskArrivalTime.count() << "s)\n";
    }
    else {
        cout << "Процессор свободен\n";
    }
    cout << "----------------------------------------\n";
}

void executeTask(Task& currentTask, vector<Task>& f0, vector<Task>& f1,
vector<Task>& f2, vector<Task>& S, bool& taskFromStack,
chrono::time_point<chrono::steady_clock> startTime) {
    while (true) {
        this_thread::sleep_for(chrono::seconds(15));
        {
            lock_guard<mutex> lock(mtx);
            chrono::time_point<chrono::steady_clock> currentTime =
chrono::steady_clock::now();
            chrono::duration<double> elapsedTime = currentTime - startTime;

            cout << "Задача выполнена: " << currentTask.name << endl;

            if (!f0.empty()) {
```

```
                Task task = f0.front();
                chrono::duration<double> taskArrivalTime = task.arrivalTime -
startTime;
            }
            else if (!f1.empty()) {
                Task task = f1.front();
                chrono::duration<double> taskArrivalTime = task.arrivalTime -
startTime;
            }
            else if (!f2.empty()) {
                Task task = f2.front();
                chrono::duration<double> taskArrivalTime = task.arrivalTime -
startTime;
            }
            else if (!S.empty()) {
                Task task = S.back();
                chrono::duration<double> taskArrivalTime = task.arrivalTime -
startTime;
            }
        }
        currentTask.priority = -1;

        if (!f0.empty()) {
            currentTask = f0.front();
            f0.erase(f0.begin());
        }
        else if (!f1.empty()) {
            currentTask = f1.front();
            f1.erase(f1.begin());
        }
        else if (!f2.empty()) {
            currentTask = f2.front();
            f2.erase(f2.begin());
        }
        else if (!S.empty()) {
            Task stackTask = S.back();
            S.pop_back();
            taskFromStack = true;
            currentTask = stackTask;
        }
        else {
            taskFromStack = false;
            currentTask.priority = -1;
            break;
        }

        displayState(f0, f1, f2, S, currentTask, startTime);
    }
}

int main() {
    setlocale(LC_ALL, "Russian");
    vector<Task> f0, f1, f2;
    vector<Task> S;
    Task currentTask(-1, "", chrono::steady_clock::now());
    bool taskFromStack = false;

    chrono::time_point<chrono::steady_clock> startTime =
chrono::steady_clock::now();
    thread stackThread(executeTask, ref(currentTask), ref(f0), ref(f1), ref(f2),
ref(S), ref(taskFromStack), startTime);
    stackThread.detach();

    while (true) {
```

```
        cout << "Выберите действие (0 - добавить задачу в f0, 1 - добавить задачу в
f1, 2 - добавить задачу в f2, 3 - создать случайную задачу, 9 - завершить): \n";
        int choice;
        cin >> choice;

        if (choice == 0 || choice == 1 || choice == 2) {
            if (f0.size() + f1.size() + f2.size() + S.size() >= 3 * MaxTasks) {
                cout << "Все очереди и стек полностью заполнены. Невозможно добавить
новую задачу.\n";
            }
            else {
                string taskName;
                cout << "Введите имя задачи: ";
                cin.ignore();
                getline(cin, taskName);

                chrono::time_point<chrono::steady_clock> currentTime =
chrono::steady_clock::now();
                if (choice == 0 && f0.size() < MaxTasks) {
                    f0.push_back(Task(0, taskName, currentTime));
                }
                else if (choice == 1 && f1.size() < MaxTasks) {
                    f1.push_back(Task(1, taskName, currentTime));
                }
                else if (choice == 2 && f2.size() < MaxTasks) {
                    f2.push_back(Task(2, taskName, currentTime));
                }
                else {
                    cout << "Очередь полностью заполнена. Задача не добавлена.\n";
                }

                if (!taskFromStack) {
                    if (currentTask.priority == -1 || currentTask.priority > choice)
{
                        if (currentTask.priority != -1) {
                            S.push_back(currentTask);
                        }
                        currentTask = (choice == 0) ? f0.front() : (choice == 1) ?
f1.front() : f2.front();
                        if (choice == 0 && !f0.empty()) {
                            f0.erase(f0.begin());
                        }
                        else if (choice == 1 && !f1.empty()) {
                            f1.erase(f1.begin());
                        }
                        else if (choice == 2 && !f2.empty()) {
                            f2.erase(f2.begin());
                        }
                    }
                }
            }
        }
        else if (choice == 3) {
            if (f0.size() + f1.size() + f2.size() + S.size() >= 3 * MaxTasks) {
                cout << "Все очереди и стек полностью заполнены. Невозможно создать
случайную задачу.\n";
            }
            else {
                random_device rd;
                mt19937 gen(rd());
                uniform_int_distribution<int> priorityDist(0, 2);
                uniform_int_distribution<int> nameDist(1, 20);
                int randomPriority = priorityDist(gen);
                int randomName = nameDist(gen);
                string taskName = to_string(randomName);
```

```
                chrono::time_point<chrono::steady_clock> currentTime =
chrono::steady_clock::now();
                if (randomPriority == 0 && f0.size() < MaxTasks) {
                    f0.push_back(Task(0, taskName, currentTime));
                }
                else if (randomPriority == 1 && f1.size() < MaxTasks) {
                    f1.push_back(Task(1, taskName, currentTime));
                }
                else if (randomPriority == 2 && f2.size() < MaxTasks) {
                    f2.push_back(Task(2, taskName, currentTime));
                }
                else {
                    cout << "Очередь полностью заполнена. Случайная задача не
добавлена.\n";
                }

                if (!taskFromStack) {
                    if (currentTask.priority == -1 || currentTask.priority >
randomPriority) {
                        if (currentTask.priority != -1) {
                            S.push_back(currentTask);
                        }
                        currentTask = (randomPriority == 0) ? f0.front() :
(randomPriority == 1) ? f1.front() : f2.front();
                        if (randomPriority == 0 && !f0.empty()) {
                            f0.erase(f0.begin());
                        }
                        else if (randomPriority == 1 && !f1.empty()) {
                            f1.erase(f1.begin());
                        }
                        else if (randomPriority == 2 && !f2.empty()) {
                            f2.erase(f2.begin());
                        }
                    }
                }
            }
        }
        else if (choice == 9) {
            cout << "Программа завершена\n";
            break;
        }
        else {
            cout << "Недопустимый выбор. Пожалуйста, выберите 0, 1, 2, 3 или 9
(завершить).\n";
        }

        displayState(f0, f1, f2, S, currentTask, startTime);
    }

    return 0;
}
```

## 3)Примеры

```
C:\MyProjectsVS\SAOD\steki_
Выберите действие (0 - добавить задачу в f0, 1 - добавить задачу в f1, 2 - добавить задачу в f2, 3 - создать случайную задачу, 9 - завершить):
3
Очередь f0 (высший приоритет):
Очередь f1 (средний приоритет):
Очередь f2 (самый низкий приоритет): 9(5.88737s)
Стек S:
Выполняется задача: 6 (приоритет 1, поступила через 5.6003s)
----------
Выберите действие (0 - добавить задачу в f0, 1 - добавить задачу в f1, 2 - добавить задачу в f2, 3 - создать случайную задачу, 9 - завершить):
3
Очередь f0 (высший приоритет):
Очередь f1 (средний приоритет): 11(6.24566s)
Очередь f2 (самый низкий приоритет): 9(5.88737s)
Стек S:
Выполняется задача: 6 (приоритет 1, поступила через 5.6003s)
----------
Выберите действие (0 - добавить задачу в f0, 1 - добавить задачу в f1, 2 - добавить задачу в f2, 3 - создать случайную задачу, 9 - завершить):
0
Введите имя задачи: 1
Очередь f0 (высший приоритет):
Очередь f1 (средний приоритет): 11(6.24566s)
Очередь f2 (самый низкий приоритет): 9(5.88737s)
Стек S: 6(5.6003s)
Выполняется задача: 1 (приоритет 0, поступила через 10.8912s)
----------
Выберите действие (0 - добавить задачу в f0, 1 - добавить задачу в f1, 2 - добавить задачу в f2, 3 - создать случайную задачу, 9 - завершить):
Задача выполнена: 1
Очередь f0 (высший приоритет):
Очередь f1 (средний приоритет):
Очередь f2 (самый низкий приоритет): 9(5.88737s)
Стек S: 6(5.6003s)
Выполняется задача: 11 (приоритет 1, поступила через 6.24566s)
----------
```

```
Выберите действие (0 - добавить задачу в f0, 1 - добавить задачу в f1, 2 - добавить задачу в f2, 3 - создать случайную задачу, 9 - завершить):
Задача выполнена: 1
Очередь f0 (высший приоритет):
Очередь f1 (средний приоритет):
Очередь f2 (самый низкий приоритет): 9(5.88737s)
Стек S: 6(5.6003s)
Выполняется задача: 11 (приоритет 1, поступила через 6.24566s)
----------
Задача выполнена: 11
Очередь f0 (высший приоритет):
Очередь f1 (средний приоритет):
Очередь f2 (самый низкий приоритет):
Стек S: 6(5.6003s)
Выполняется задача: 9 (приоритет 2, поступила через 5.88737s)
----------
Задача выполнена: 9
Очередь f0 (высший приоритет):
Очередь f1 (средний приоритет):
Очередь f2 (самый низкий приоритет):
Стек S:
Выполняется задача: 6 (приоритет 1, поступила через 5.6003s)
----------
```

## 4)Вывод:

Данный код моделирует работу с очередями задач различного приоритета, стеком и многозадачностью. В программе есть три очереди с задачами, разделенными по приоритетам, и стек для временного хранения задач более низкого приоритета. Задачи добавляются в очереди в зависимости от выбора пользователя, и их выполнение управляется в порядке убывания приоритетов. Задачи с более высоким приоритетом имеют предпочтение, но могут быть вытеснены задачами из стека, если такие имеются. Пользователь может добавлять задачи, создавать случайные задачи и завершать программу, а состояние системы отображается в реальном времени. Этот код служит для демонстрации принципов работы с планированием задач и приоритетами.